

SubTrack — Project Documentation

Purpose of this document: This is a "come back after 6 months" guide for SubTrack.

>

It explains the project in simple language: what it does, how the pieces fit together, where the important code lives, how a request travels through the application, how authentication works, how subscriptions and payments work, how to run the project, and what to check when something breaks.

1. Start Here — What Is SubTrack?

SubTrack is a full-stack web application for managing subscriptions.

Think of it like a personal subscription manager:

- A user creates an account.
- The user logs in.
- The user can add subscriptions such as Netflix, Spotify, AWS, gym membership, etc.
- Each subscription has a price, category, billing cycle, status, and next billing date.
- The user can make a simulated payment from their application balance.
- Payment history is stored.
- Notifications can be created and marked as read.
- Admin users can see broader application data.
- Worker users are intended for internal subscription/payment/notification processing.

The application has two main parts:

```
SubTrack
■
■■■ Client → React + TypeScript
■
■■■ Server → ASP.NET Core Web API + EF Core + SQL Server
```

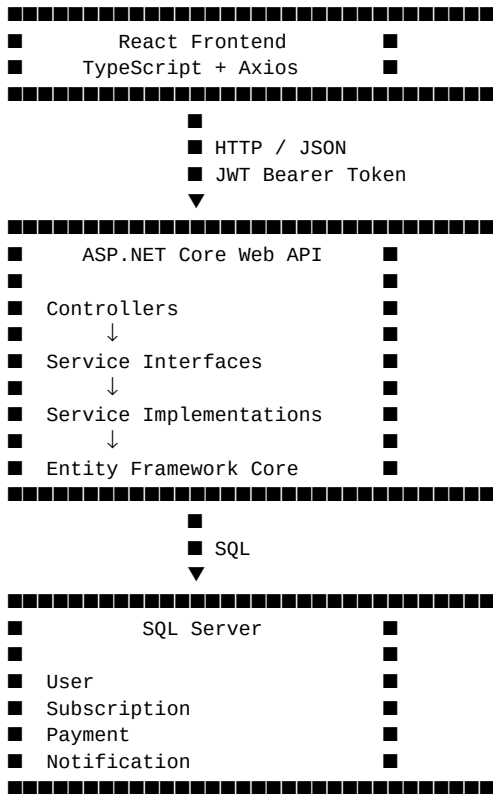
The important thing to remember is:

The main branch is a single backend application, not a microservices architecture.

The backend is organized into logical areas such as Authentication, Subscriptions, Payments, and Notifications, but they all run inside the same ASP.NET Core application.

2. The Big Picture

When you forget how the project works, remember this diagram:



A normal request therefore looks like:

```

React
↓
Axios
↓
Controller
↓
Service
↓
EF Core
↓
SQL Server
↓
EF Core
↓
Service
↓
Controller
↓
JSON response
↓
React
  
```

3. Repository and Branches

The GitHub repository used for this working copy is:

<https://github.com/chauhan12harsh/Subtrack>

The main branch is the simple/monolithic version of the project.

Important branches that have existed in the project include:

- `main` — simple/monolithic application and the primary portfolio version.
- `microservice` — more advanced microservices version.
- `monolithic-architecture` — another architecture-related branch.

Which branch should you understand first?

Start with:

`main`

This document describes the code in the **main branch**.

Do not assume that code from the `microservice` branch exists in `main`.

4. Project Folder Structure

The repository is approximately organized like this:

```
Subtrack/
├── Client/
│   ├── src/
│   │   ├── Components/
│   │   ├── Config/
│   │   ├── Layouts/
│   │   ├── Pages/
│   │   ├── Routes/
│   │   ├── Services/
│   │   ├── Types/
│   │   └── Utils/
│   ├── package.json
│   └── vite.config.ts
├── Server/
│   ├── Controllers/
│   ├── Data/
│   ├── Dtos/
│   ├── Entities/
│   ├── Enums/
│   ├── Exceptions/
│   ├── Interfaces/
│   ├── Middlewares/
│   ├── Migrations/
│   ├── Services/
│   ├── Program.cs
│   ├── Server.csproj
│   └── appsettings.example.json
└── README.md
```

The easiest way to remember the backend folders is:

Folder • Simple meaning

Controllers • Receives HTTP requests

Services • Contains business rules

Interfaces • Defines what services can do

Entities • Represents database tables

Dtos • Defines data sent between frontend and backend

Data • Database connection/context

Enums • Fixed choices such as roles/statuses

Exceptions • Custom application errors

Middlewares • Code that runs around requests, especially error handling

Migrations • EF Core database change history

5. Backend Architecture in Simple Terms

The backend follows a layered approach.

Controller

A controller is the "reception desk".

It receives the request and decides which service should handle it.

Example:

```
POST /subscription/create
  ↓
SubscriptionController
  ↓
SubscriptionService
```

Controllers should mainly deal with HTTP-related work.

Service

A service is where the actual application rules live.

For example:

```
SubscriptionService
```

knows things such as:

- a user should not create the same subscription twice
- a subscription belongs to a particular user
- a subscription can be paused/cancelled/active
- a subscription can be renewed

- a next billing date needs to be changed during renewal

Interface

An interface describes what a service can do without describing how it does it.

For example:

```
ISubscriptionService
```

is the contract.

```
SubscriptionService
```

is the implementation.

This allows the controller to depend on the contract rather than directly depending on the implementation.

Entity

An entity represents information stored in SQL Server.

The four important entities are:

```
User  
Subscription  
Payment  
Notification
```

DTO

DTO means **Data Transfer Object**.

It is the shape of data that enters or leaves the API.

For example, the frontend does not need to send an entire User database object just to log in.

It sends:

```
Email  
Password
```

using `UserLoginDto`.

DTOs help keep API data separate from database entities.

6. Application Startup — Program.cs

The most important backend file to understand first is:

```
Server/Program.cs
```

This file sets up the application.

It currently configures:

1. Controllers
2. SQL Server / EF Core
3. Dependency Injection
4. JSON enum conversion
5. JWT authentication
6. CORS
7. Middleware
8. Authorization
9. Controller routing

The service registrations are:

```
IAuthService      → AuthService
IUserService       → UserService
ISubscriptionService → SubscriptionService
IPaymentService    → PaymentService
INotificationService → NotificationService
```

This means:

```
Controller asks for ISubscriptionService
      ↓
ASP.NET gives it SubscriptionService
```

You do not manually create the service inside every controller.

7. Dependency Injection — The Simple Explanation

Dependency Injection sounds complicated but the idea is simple.

Instead of a controller doing this:

```
new SubscriptionService(...)
```

ASP.NET creates the service and gives it to the controller.

Why?

Because it keeps the classes loosely connected.

The application uses:

```
builder.Services.AddScoped<ISubscriptionService, SubscriptionService>();
```

Scoped means one service instance is normally used within one HTTP request scope.

8. Database — ApplicationDbContext

The database context is:

```
Server/Data/ApplicationDbContext.cs
```

It is the bridge between C# code and SQL Server.

It exposes:

```
DbSet<User>  
DbSet<Subscription>  
DbSet<Payment>  
DbSet<Notification>
```

You can think of each DbSet as a table-like collection.

C#		SQL Server
context.User	→	User
context.Subscription	→	Subscription
context.Payment	→	Payment
context.Notification	→	Notification

The application uses Entity Framework Core to translate C# queries into SQL.

9. Database Entities

9.1 User

File:

```
Server/Entities/User.cs
```

A user contains:

```
Id  
Username  
Email  
PasswordHash
```

Balance
CreatedAt
Role

Important:

The password is not stored as plain text.

The project uses ASP.NET Core's PasswordHasher<User> to hash and verify passwords.

9.2 Subscription

File:

Server/Entities/Subscription.cs

A subscription contains:

Id
UserId
Name
Amount
Category
Status
BillingCycle
NextBillingDate

Example:

Name: Spotify
Amount: 119
Category: Entertainment
Status: Active
BillingCycle: Monthly
NextBillingDate: 2026-10-01

User Id tells us which user owns the subscription.

9.3 Payment

File:

Server/Entities/Payment.cs

A payment contains:

Id
UserId
SubscriptionId
Amount
Status
PaymentDate
TransactionReference

The transaction reference looks like:

```
TXN + GUID
```

Example:

```
TXN2f5c...
```

9.4 Notification

File:

```
Server/Entities/Notification.cs
```

A notification contains:

```
Id
UserId
Title
Message
Type
IsRead
CreatedAt
```

The important field for the UI is:

```
IsRead
```

This is used for unread notification counts and marking notifications as read.

10. Enums

Enums are used when a value should come from a known list.

UserRole

```
User
Admin
Worker
```

BillingCycle

```
Monthly
Quarterly
HalfYearly
Yearly
```

BillingStatus

Active
Paused
Cancelled

PaymentStatus

Pending
Processing
Completed
Failed
Refunded

NotificationType

PaymentSuccess
PaymentFailed
SubscriptionRenewed
SubscriptionPaused
SubscriptionCancelled
General

The application configures these enums to be stored as strings in SQL Server rather than numeric values.

For example:

"Active"

instead of:

0

This makes the database easier for a human to understand.

11. Authentication — How Login Works

Authentication answers:

"Who are you?"

The flow is:

```
User enters email + password
↓
POST /auth/login
↓
AuthController
↓
AuthService
↓
Find user by email
↓
Verify password hash
↓
Create JWT
↓
Return token
```

The JWT contains important claims:

```
NameIdentifier = User ID
Role           = User/Admin/Worker
```

The token is valid for approximately one day.

12. JWT Configuration

The backend expects these settings:

```
"TokenDetails": {
  "SigningKey": "<Token_signing_key>",
  "Issuer": "<issuer>",
  "Audience": "<audience>"
}
```

These values are used when:

1. Creating the JWT.
2. Validating the JWT on later requests.

The signing key is especially important.

Never commit a real secret signing key to GitHub.

13. Authorization — What Can Each Role Do?

Authorization answers:

"Now that we know who you are, are you allowed to do this?"

There are three roles.

User

Normal application user.

Can:

- view/update own profile
- change own password
- create subscriptions
- update own subscriptions
- view own subscriptions

- update own subscription status
- process payments
- view own payments
- view own notifications
- mark own notifications as read

Admin

Admin has additional access.

Examples:

- view all users
- change user roles
- view all subscriptions
- delete subscriptions
- view all payments
- delete notifications
- view all notifications
- access subscriptions due for renewal

Worker

Worker represents internal processing functionality.

Examples:

- renew subscriptions
- process internal payments
- create notifications

The idea is that worker-only endpoints could later be called by a scheduled job/background process.

Currently, the worker role is represented through authorization attributes; the project does not yet contain a full background worker service.

14. Frontend Authentication

The frontend creates an Axios client here:

Client/src/Services/BaseApi.ts

The API base URL is currently:

`https://localhost:7136/`

Before every request, the Axios interceptor checks:

`localStorage → accessToken`

If the token exists, it adds:

`Authorization: Bearer <token>`

Therefore:

```
Login
↓
JWT returned
↓
Frontend stores accessToken
↓
Axios automatically sends token
↓
Backend authenticates request
```

If authentication suddenly stops working, check the token in browser local storage and check the backend HTTPS URL first.

15. User Management

Important backend files:

Controllers/UserController.cs
Services/UserService.cs

Available operations include:

```
GET    /user/profile
PATCH /user/update
PATCH /user/changepassword
GET    /user/alluser
PATCH /user/role/{userId}/{role}
```

The first three are for the logged-in user.

The last two require Admin authorization.

16. Subscription Management

Important files:

```
Controllers/SubscriptionController.cs
Services/SubscriptionService.cs
Dtos/Subscription/
Entities/Subscription.cs
```

The main subscription flow is:

```
Create
↓
Store subscription
↓
User can view/update it
↓
Status can change
↓
When billing is due
↓
Payment can be processed
↓
Subscription can be renewed
↓
NextBillingDate moves forward
```

17. Creating a Subscription

Endpoint:

```
POST /subscription/create
```

The user sends:

```
Name
Amount
Category
BillingCycle
NextBillingDate
```

The service first checks whether the same user already has a subscription with the same:

```
Name + Category
```

If yes:

```
AlreadyExistsException
```

If no:

```
Create subscription
↓
Save to database
↓
Return subscription DTO
```

18. Updating a Subscription

Endpoint:

```
PATCH /subscription/update/{subscriptionid}
```

The service first checks:

```
Does this subscription exist?  
AND  
Does it belong to this user?
```

This is important.

A user should not be able to modify somebody else's subscription simply by knowing its ID.

Only supplied fields are changed.

For example:

```
Name = null  
Amount = 500  
Category = null
```

means:

```
Keep old name  
Change amount  
Keep old category
```

19. Subscription Status

Endpoint:

```
PUT /subscription/status/{subscriptionid}/{status}
```

Possible statuses:

```
Active  
Paused  
Cancelled
```

The service checks whether the status is already the requested value.

If it is already the same:

```
AlreadyUpdatedException
```

Otherwise it changes the status and saves it.

20. Getting User Subscriptions

Endpoint:

```
GET /subscription/user-subscription
```

The backend:

1. Gets the logged-in user's ID from the JWT.
2. Finds subscriptions owned by that user.
3. Excludes cancelled subscriptions.
4. Sorts them by next billing date.
5. Returns them.

This is an important ownership pattern used throughout the project.

21. Subscription Renewal

Renewal happens through:

```
PATCH /subscription/renew/{subscriptionid}
```

This endpoint requires:

```
Worker
```

role.

The service checks:

```
Does subscription exist?  
↓  
Is it cancelled?  
↓  
Is it active?  
↓  
What is the billing cycle?  
↓  
Move NextBillingDate forward
```

Important current implementation detail

The entity supports:

```
Monthly  
Quarterly  
HalfYearly  
Yearly
```

However, the current `RenewSubscription` implementation explicitly handles:

```
Monthly  
Yearly
```

and throws an error for other cycles.

So if you return to this project later and want full renewal support, this is one of the first places to inspect.

A likely future improvement would be:

```
Monthly      → AddMonths(1)
Quarterly    → AddMonths(3)
HalfYearly   → AddMonths(6)
Yearly       → AddYears(1)
```

22. Finding Due Subscriptions

Endpoint:

```
GET /subscription/due
```

This endpoint is available to:

```
Worker
Admin
```

It looks for subscriptions where:

```
Status == Active
AND
NextBillingDate.Date == today
```

This is intended to identify subscriptions that need billing/renewal processing.

Important:

The current project exposes the endpoint, but it does not contain a complete automated scheduler/background worker that continuously calls it.

23. Payment System

Important files:

```
Controllers/PaymentController.cs
Services/PaymentService.cs
Entities/Payment.cs
Dtos/Payment/
```

The payment system is currently a **simulation**, not a real Razorpay/Stripe integration.

The code even contains a comment explaining that a real external payment gateway can be connected later.

24. Payment Flow

The main payment endpoint is:

```
POST /payment/process
```

The flow is:

```
graph TD
    A[User requests payment] --> B[Find user]
    B --> C[Check balance]
    C --> D[Generate payment status]
    D --> E[Create payment record]
    E --> F[If Completed:]
    F --> G[Deduct user balance]
    G --> H[Renew subscription]
    H --> I[Save payment]
    I --> J[Return payment response]
```

25. Payment Status Simulation

Currently the service uses:

```
Random.Shared.Next(0, 4)
```

and converts the result into a `PaymentStatus`.

That means payment status is simulated randomly among the first four enum values:

```
Pending
Processing
Completed
Failed
```

Refunded is not generated by this random range.

This is important to remember because this is **not a real payment gateway**.

If you later integrate Stripe/Razorpay, this part of `PaymentService` is the main area that would change.

26. Balance Handling

Before processing payment:

```
user.Balance < payment amount?
```

If yes:

```
InsufficientBalanceException
```

which becomes:

```
HTTP 402 Payment Required
```

If payment status is Completed:

```
user.Balance
  ↓
user.Balance - payment amount
```

The updated balance is then saved with the payment.

27. Payment Ownership

For user payment lookup, the service uses both:

```
Payment ID
AND
Authenticated User ID
```

For example:

```
payment.Id == paymentid
AND
payment.UserId == userId
```

This is another security/ownership check.

It prevents a user from simply changing a payment ID and reading somebody else's payment.

28. Internal Payment Endpoint

Endpoint:

```
POST /payment/processinternal
```

This endpoint requires:

```
Worker
```

It accepts a `WorkerProcessPaymentDto`, which includes:

```
UserId
SubscriptionId
```

Amount

The worker can therefore process a payment on behalf of a user.

29. Notifications

Important files:

```
Controllers/NotificationController.cs
Services/NotificationService.cs
Entities/Notification.cs
```

Notifications belong to users.

A notification has:

```
Title
Message
Type
IsRead
CreatedAt
```

30. Notification Operations

Create

```
POST /notification/create
```

Worker-only.

Get current user's notifications

```
GET /notification/my
```

Mark one as read

```
PATCH /notification/read/{notificationId}
```

Mark all as read

```
PATCH /notification/readall
```

Count unread notifications

```
GET /notification/unreadcount
```

Admin delete

```
DELETE /notification/delete/{notificationId}
```

Admin list all

```
GET /notification/all
```

31. Important Notification Behavior

When marking a notification as read:

```
Does notification exist for this user?
↓
No → NotFoundException
↓
Yes
↓
Already read?
↓
Yes → AlreadyUpdatedException
↓
No
↓
Set IsRead = true
```

This is why the frontend can distinguish between:

```
notification does not exist
```

and:

```
notification was already read
```

32. Exception Handling

The project has a global exception middleware:

```
Server/Middlewares/GlobalExceptionMiddleware.cs
```

This means services can throw application-specific exceptions and the middleware converts them into HTTP responses.

Examples:

```
BadRequestException
↓
400

NotFoundException
↓
404

AlreadyExistsException
↓
409
```

```
AlreadyUpdatedException
  ↓
409

InsufficientBalanceException
  ↓
402

Unexpected Exception
  ↓
500
```

The middleware also logs unexpected exceptions.

33. Custom Exceptions

Look in:

```
Server/Exceptions/
```

and related authentication exception folders.

Important exceptions include:

```
BadRequestException
NotFoundException
AlreadyExistsException
AlreadyUpdatedException
InsufficientBalanceException
UnauthorizedException
JwtConfigurationException
```

When adding a new business rule, first check whether an existing exception already fits.

Do not create a new exception for every tiny situation.

34. One Important Exception Detail

The current middleware maps:

```
UnauthorizedException
```

to:

```
404 Not Found
```

This is unusual.

Normally an authentication/authorization-related error might use:

```
401 Unauthorized
```

or:

```
403 Forbidden
```

depending on the situation.

If you later revisit error handling, inspect this mapping.

35. Frontend Structure

The React application is under:

```
client/src/
```

Important folders:

```
Components/  
Pages/  
Layouts/  
Routes/  
Services/  
Types/  
Utils/  
Config/
```

Think of them as:

Folder • Meaning

Pages • Large screens/pages

Components • Reusable UI pieces

Layouts • Common page structure

Routes • Which page opens for which URL

Services • Calls the backend

Types • TypeScript data shapes

Utils • Small helper functions

Config • Frontend configuration

36. Frontend Services

The frontend has API service code for:

```
Auth  
Notification  
Payment  
Subscription
```

There are also API client files and service files.

The basic pattern is:

```
React component
  ↓
Frontend service
  ↓
Axios
  ↓
ASP.NET API
```

This is preferable to putting API calls directly everywhere inside React components.

37. Frontend API Base URL

Current file:

```
Client/src/Services/BaseApi.ts
```

Current API URL:

```
https://localhost:7136/
```

If the backend starts on another port, frontend requests will fail until this is updated.

This should be one of the first things to check when the frontend says:

```
Network Error
```

or:

```
Failed to fetch
```

38. Frontend Technology

Current package.json shows:

```
React 19
TypeScript 6
Vite
Axios
React Hook Form
React Router
Tailwind CSS
Recharts
```

Useful commands:

```
npm install
npm run dev
npm run build
npm run lint
npm run preview
```

39. Backend Technology

The backend currently targets:

```
.NET 10
```

Important packages include:

```
Microsoft.AspNetCore.Authentication.JwtBearer
Microsoft.EntityFrameworkCore
Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore.Tools
```

40. How To Run The Project After 6 Months

When returning to the project, follow this exact order.

Step 1 — Clone / pull

```
git clone https://github.com/chauhan12harsh/Subtrack.git
cd Subtrack
```

If already cloned:

```
git pull
```

Then make sure you are on:

```
main
```

Step 2 — Check the backend SDK

Run:

```
dotnet --version
```

The project targets:

```
net10.0
```

Step 3 — Check Node

Run:

```
node --version
npm --version
```

Step 4 — Configure backend settings

Look at:

```
Server/appsettings.example.json
```

You need local values for:

```
ConnectionStrings:DefaultConnection
TokenDetails:SigningKey
TokenDetails:Issuer
TokenDetails:Audience
```

Do not put real secrets into Git.

Step 5 — Restore backend

From the repository root:

```
dotnet restore Server/Server.csproj
```

Step 6 — Update database

```
dotnet ef database update --project Server/Server.csproj
```

If EF CLI is not available, check/install the appropriate `dotnet-ef` tool.

Step 7 — Start backend

```
dotnet run --project Server/Server.csproj
```

For HTTPS:

```
dotnet run --project Server/Server.csproj --launch-profile https
```

Step 8 — Check the frontend API URL

Open:

```
Client/src/Services/BaseApi.ts
```

Make sure:

```
https://localhost:7136/
```

matches the backend URL.

Step 9 — Install frontend packages

```
cd Client  
npm install
```

Step 10 — Start frontend

```
npm run dev
```

41. If The Project Does Not Start

Use this checklist.

Backend won't start

Check:

```
.NET SDK  
SQL Server  
connection string  
JWT configuration  
HTTPS certificate
```

Run:

```
dotnet build Server/Server.csproj
```

Look at the first meaningful compiler error rather than the last cascade of errors.

Database error

Check:

```
Server/appsettings/local configuration
SQL Server running
database exists
connection string correct
```

Then:

```
dotnet ef database update --project Server/Server.csproj
```

Frontend cannot call backend

Check:

```
Client/src/Services/BaseApi.ts
```

Verify the API URL.

Then check:

```
Backend running?
HTTPS trusted?
CORS?
Browser Network tab?
```

Login works but later requests fail

Check:

```
localStorage
```

for:

```
accessToken
```

Then inspect the browser request headers.

The request should contain:

```
Authorization: Bearer <token>
```

42. How To Debug A Request

When something is broken, do not immediately jump between files.

Trace the request from front to back.

Example:

```
User clicks "Pay"
↓
Payment.tsx
↓
paymentService.ts
↓
paymentApiClient.ts
↓
Axios
↓
POST /payment/process
↓
PaymentController
↓
IPaymentService
↓
PaymentService
↓
ApplicationDbContext
↓
SQL Server
```

If you follow this path, you will usually find where the problem is.

43. How To Debug Authentication

Follow:

```
Login page
↓
authService
↓
POST /auth/login
↓
AuthController
↓
AuthService
↓
PasswordHasher
↓
JWT creation
↓
Token returned
```

Then:

```
JWT stored in localStorage
↓
Axios interceptor
↓
Authorization header
↓
JWT middleware
↓
Controller [Authorize]
```

44. How To Debug Subscription Problems

Start with:

```
Client/src/Services/Subscription/
```

Then:

```
SubscriptionController
↓
SubscriptionService
```

Then inspect:

```
ApplicationDbContext
Subscription entity
```

Common things to check:

```
subscription ID
user ID
status
billing cycle
next billing date
```

45. How To Debug Payment Problems

Start with:

```
Client/src/Services/Payment/
```

Then:

```
PaymentController
↓
PaymentService
```

Check:

```
User exists?
Balance sufficient?
Subscription ID correct?
Payment status?
Subscription renewal successful?
Payment saved?
```

Remember:

Payment status is currently simulated randomly.

So a payment can legitimately return a non-completed status because of the current simulation.

46. How To Debug Notification Problems

Trace:

```

Notification component
    ↓
notification service
    ↓
NotificationController
    ↓
NotificationService
    ↓
ApplicationDbContext

```

Check:

```
UserId
NotificationId
IsRead
NotificationType
CreatedAt
```

47. Database Relationships — Mental Model

Even though the entities currently use IDs rather than a large set of explicit EF navigation properties, the logical relationships are:

```
User
■
■■■■■■■■■■■■■■■■■■■■< Subscription
■
■■■■■■■■■■■■■■■■■■■■< Payment
■
■■■■■■■■■■■■■■■■■■■■< Notification

Subscription
■
■■■■■■■■■■■■■■■■■■■■< Payment
```

Meaning:

```
One user
  can have many subscriptions

One user
  can have many payments

One user
  can have many notifications

One subscription
  can have many payments
```

— — —

48. DTOs — Where To Look

When the frontend sends the wrong data shape, inspect:

```
Server/Dtos/
```

Main groups:

```
Auth/  
Notification/  
Payment/  
Subscription/  
User/
```

If an API expects:

```
CreateSubscriptionDto
```

look there first.

If an API returns:

```
SubscriptionResponseDto
```

look there.

49. Migrations

Database migration files are under:

```
Server/Migrations/
```

There is an initial migration and an EF Core model snapshot.

If you change an entity, for example:

```
Add a new field to Subscription
```

you will generally need to create a new migration and update the database.

Typical workflow:

```
dotnet ef migrations add <MigrationName> --project Server/Server.csproj  
dotnet ef database update --project Server/Server.csproj
```

Before doing this, understand whether the database already contains important local data.

50. What The Project Does NOT Currently Have

This section is important because it prevents you from remembering features that are not actually implemented.

The main branch is **not** currently a full distributed system.

Do not assume the main branch has:

- Kafka
- RabbitMQ
- Redis
- Kubernetes
- API Gateway
- Service Mesh
- Saga pattern
- Event sourcing
- CQRS
- Distributed tracing
- Outbox pattern
- Multiple independently deployed databases
- Real Stripe/Razorpay integration
- A complete automatic background worker/scheduler

Those ideas may belong to the microservice/advanced direction, but they are not the core architecture of main.

51. Current Known Limitations / Things To Remember

These are useful places to inspect if you continue development later.

1. Renewal only handles Monthly and Yearly

Although the enum contains:

```
Monthly
Quarterly
HalfYearly
Yearly
```

the current renewal implementation only handles:

```
Monthly
Yearly
```

2. Payment is simulated

Payment status is randomly generated.

It is not connected to a real payment provider.

3. Automatic background processing is not complete

There is a worker role and endpoints for due subscriptions/internal processing, but the current main branch does not contain a complete automated background worker that continuously runs those operations.

4. Some notification creation code in PaymentService is commented out

The payment service has a planned notification integration, but the relevant creation call is currently commented.

So do not assume a successful payment automatically creates a notification just because notification functionality exists.

5. Error mappings should be reviewed

UnauthorizedException is currently mapped to 404 in the global middleware.

This may be worth revisiting later.

6. User profile update can change balance

The current UpdateUserProfile accepts a Balance value.

This means the current API design allows balance to be changed through the profile update path.

For a real financial application, balance should normally be controlled by payment/business operations rather than an ordinary profile update.

This is an important future security/business-rule improvement.

7. CORS is currently open

The current backend CORS policy uses:

```
AllowAnyOrigin()  
AllowAnyHeader()  
AllowAnyMethod()
```

This is convenient for development but should be restricted to known frontend origins for a production deployment.

52. Recommended Reading Order When You Return

If you forget everything, do not read the entire project from top to bottom.

Read these files in this order:

1. README.md
2. Server/Program.cs
3. Server/Data/ApplicationDbContext.cs
4. Server/Entities/
 - User.cs
 - Subscription.cs
 - Payment.cs
 - Notification.cs
5. Server/Enums/
6. Server/Controllers/
7. Server/Interfaces/
8. Server/Services/
9. Server/Middlewares/GlobalExceptionMiddleware.cs
10. Server/Dtos/
11. Client/src/Services/
12. Client/src/Components/
13. Client/src/Pages/

This order goes from:

```
Big picture  
↓  
Application setup  
↓  
Database  
↓  
Data model  
↓  
API  
↓  
Business logic  
↓  
Frontend communication  
↓  
UI
```

53. If You Want To Add A New Feature

Use this pattern.

Suppose you want to add:

Subscription reminder

Think:

Step 1 — What data is needed?

Maybe:

Reminder date
Reminder message

Would this require a new entity/table?

Step 2 — What API is needed?

Maybe:

POST /notification/reminder
GET /notification/reminders

Step 3 — DTO

Create DTOs if the request/response needs a specific data shape.

Step 4 — Interface

Add the required method to the appropriate interface.

Step 5 — Service

Put business rules in the service.

Step 6 — Controller

Expose the HTTP endpoint.

Step 7 — Database

If the entity changes:

```
Create migration
Update database
```

Step 8 — Frontend

Add:

```
API client
service
types
component/UI
```

Step 9 — Test

Test:

```
happy path
invalid input
unauthorized access
wrong user's resource
duplicate data
not found
edge cases
```

54. How To Think About Ownership Checks

This is one of the most important security ideas in the project.

Imagine:

```
User A
Subscription ID = ABC
```

User B should not be able to request:

```
GET /subscription/ABC
```

and receive User A's data.

The service should check:

```
subscription.Id == ABC
AND
subscription.UserId == currentUserId
```

This pattern appears in the project for user-owned resources.

Whenever you add a new endpoint that works with a user's data, ask:

"Am I checking the authenticated user's ID as well as the resource ID?"

55. HTTP Status Code Mental Model

Remember these basics:

```
200 OK
    Request worked.

400 Bad Request
    Request/data is invalid.

401 Unauthorized
    Authentication is missing/invalid.

403 Forbidden
    User is authenticated but does not have permission.

404 Not Found
    Requested resource does not exist.

409 Conflict
    Request conflicts with current state, such as duplicate data.

402 Payment Required
    Used here for insufficient balance.

500 Internal Server Error
    Unexpected server-side problem.
```

The exact mapping in the current code should be checked before changing behavior.

56. Git Workflow For Future You

Before starting work:

```
git status
git branch
git pull
```

Create a feature branch when appropriate:

```
git checkout -b feature/<feature-name>
```

Work in small changes.

Then:

```
git status
git diff
```

Commit with a meaningful message:

```
git add .
git commit -m "feat: add subscription reminders"
```

Push:

```
git push origin feature/<feature-name>
```

Do not blindly commit:

```
appsettings.json
passwords
JWT signing keys
connection strings
```

57. Useful Git Commands

Check current branch:

```
git branch
```

Check changed files:

```
git status
```

See actual changes:

```
git diff
```

Get latest changes:

```
git pull
```

See recent commits:

```
git log --oneline --decorate -10
```

Switch to main:

```
git checkout main
```

58. API Testing

Postman is useful for testing the backend without using the React UI.

A good testing sequence is:

1. Register
2. Login
3. Copy JWT
4. Add Bearer token
5. Get profile
6. Create subscription
7. Get subscriptions
8. Update subscription
9. Process payment
10. Check payment history
11. Check notifications

For protected endpoints:

Authorization: Bearer <JWT>

For Admin endpoints, the logged-in user must have:

Role = Admin

For Worker endpoints:

Role = Worker

59. A Practical "I Forgot Everything" Restart Checklist

When you return months later:

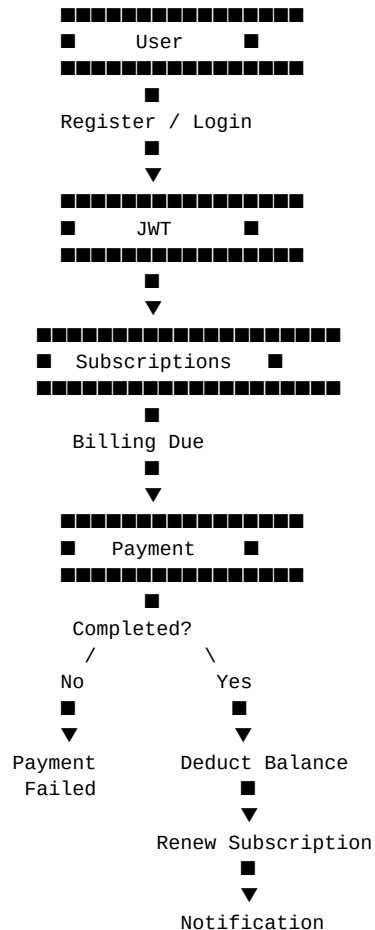
- [] Open this document
- [] Open README.md
- [] Checkout main
- [] Pull latest code
- [] Check .NET version
- [] Check Node/npm
- [] Start SQL Server
- [] Configure local app settings
- [] Run EF database update
- [] Start backend
- [] Check backend URL
- [] Check Client/src/Services/BaseApi.ts
- [] npm install
- [] npm run dev
- [] Register/login
- [] Open browser developer tools
- [] Verify JWT is stored
- [] Verify API requests work

Then start reading:

```
Program.cs
  ↓
ApplicationDbContext
  ↓
Entities
  ↓
Controllers
  ↓
Services
```

60. Core Business Flow — The One Diagram To Remember

If you remember only one business diagram, remember this:



This is the heart of the application.

61. Quick File Map

When you have a problem, start here:

Problem • First file/folder to open

Application won't start • Server/Program.cs

Database problem • Server/Data/ApplicationDbContext.cs

Login problem • Authentication/Controllers, Authentication/Services

JWT problem • Program.cs, AuthService.cs

User problem • UserController.cs, UserService.cs

Subscription problem • SubscriptionController.cs, SubscriptionService.cs

Payment problem • PaymentController.cs, PaymentService.cs

Notification problem • NotificationController.cs, NotificationService.cs

HTTP error handling • GlobalExceptionHandler.cs

Database structure • Entities/, ApplicationDbContext.cs, Migrations/

API input/output • Dtos/

Frontend API problem • Client/src/Services/

Frontend routing • Client/src/Routes/

UI problem • Client/src/Components/, Pages/

API URL problem • Client/src/Services/BaseApi.ts

62. Final Mental Model

SubTrack is easiest to understand if you divide it into five questions:

1. Who is the user?

Authentication
JWT
User
Role

2. What does the user own?

Subscriptions
Payments
Notifications

3. What can the user do?

Authorization
User/Admin/Worker

4. Where is the business logic?

Services

5. Where is the data stored?

EF Core
↓
SQL Server

Put those five answers together and the entire architecture becomes much easier to remember.

63. Current Main-Branch Source of Truth

This document is intentionally based on the **actual source code in the main branch**, rather than relying on an older README architecture description.

When this document and the source code disagree in the future:

The current source code is the final source of truth.

When making significant architectural changes, update this document so that the next version of you does not have to reverse-engineer the project again.

64. Suggested Maintenance Rule

Whenever you add a meaningful feature, update at least these parts of the documentation:

- [] What the feature does
- [] Which backend files were added/changed
- [] Which API endpoints were added
- [] Authentication/role requirements
- [] Database changes
- [] Frontend files affected
- [] How to test it
- [] Known limitations

The goal is not to document every line of code.

The goal is to leave enough information that **future you can understand the project in 30–60 minutes instead of spending several days reverse-engineering it.**